Stdin Survival Kit

How to read an Input Format and turn it into working Python, every time, without guessing. One idea, a four-step method, three worked walkthroughs, then every input shape as a lookup table.

The ninety percent template

If you remember nothing else, remember these lines. They cover the large majority of assessment problems and you can type them from memory in under ten seconds.

```
import sys
input = sys.stdin.readline    # faster; keeps the trailing newline

n = int(input())
arr = list(map(int, input().split()))
```

The habit that finds every bug before the grader does

Put the sample input in a file called `in.txt`, then run your solution the way the grader runs it:

```
python sol.py < in.txt
```

If you never do this, you are testing by typing input by hand, which hides newline and format bugs completely. **Every practice problem, every time.**

The one idea everything else follows from

Standard input is a queue of lines. The whole input sits in front of your program in order, and each call to `input()` removes exactly one line from the front and hands it to you as a string. That line is then gone - the next call gives you the one after it.

```
stdin, before your program runs:

    +-------------+
    | 5           |  <- input() takes this one
    | 1 2 3 4 5   |  <- then this one
    +-------------+

n   = int(input())                    # consumed line 1 -> "5"
arr = list(map(int, input().split())) # consumed line 2 -> "1 2 3 4 5"
```

That is genuinely the entire concept. Everything else is a consequence of it:

— You must read the same number of lines the problem says, in the same order. Read one too few and everything after it shifts up; read one too many and you crash or get an empty string.

— A line comes to you as one string. If several values sit on that line, you take the line once and then split it - you do not call input() twice.

— input() has no idea what type you want. Everything arrives as text, so you cast it yourself.

— Reading is destructive and one-directional. You cannot go back, so if you need a value twice, store it.

One line about versions, then never again

If the language dropdown offers both **Python** and **Python 3**, pick **Python 3**. That is the whole rule. Everything in this document is Python 3.

The method: turning an Input Format into code

Every problem statement has an **Input Format** block. It is not decoration - it is a line-by-line specification of the queue, written in English. Your job is to translate it downward, one line of English into one line of Python. Do this the same way every time and you stop having to think about it.

1. **Number the lines the statement describes.** Literally count them. "The first line contains n" is line 1. "The second line contains n integers" is line 2. If it says "the next R lines", that is a loop of R lines, not one line.
2. **Write one read per line, top to bottom.** In the same order. Do not skip ahead to the interesting part - the reads must happen in the order the data arrives, because reading is destructive.
3. **Cast as you read.** int() for one number, list(map(int, ...)) for many, .strip() for a string. Decide the type from the words in the statement: "integer" means int, "word" or "string" means strip, "space-separated" means split.
4. **Echo what you parsed, before you solve anything.** Print your variables, run against the sample, confirm they hold what you expected. Then delete the echo and write the algorithm. This one habit catches almost every parsing bug in about fifteen seconds.

Step 4 in practice - the echo

Before writing a single line of algorithm:

```
n   = int(input())
arr = list(map(int, input().split()))

print(n, arr)        # run it. Do you see: 5 [1, 2, 3, 4, 5] ?
                      # yes -> delete it, solve the problem
                      # no  -> your parse is wrong, fix it now
```

Debugging a parse is trivial. Debugging an algorithm that is silently receiving the wrong data is not. **Always confirm the input before you trust it.**

And when a statement is genuinely ambiguous - or you cannot tell whether there is a count line - dump the raw stream and look at it:

```
import sys
print(repr(sys.stdin.read()))

# shows you EXACTLY what you were given, newlines and all:
#   '5\n1 2 3 4 5\n'
# now there is nothing left to guess about
```

`repr()` is the important part - it makes the newline characters visible instead of printing them. Do this once in the custom-input box and the format stops being a mystery.

Three worked walkthroughs

Read these slowly. The point is not the final code, it is the translation from the English on the left to the Python on the right.

Walkthrough 1 - a count and an array

WHAT THE STATEMENT SAYS

```
Input Format

The first line contains an
integer n, the size of the
array.
The second line contains n
space-separated integers.
```

SAMPLE INPUT

```
5
1 2 3 4 5
```

TRANSLATING IT, LINE BY LINE

"The first line contains an integer n"

```
n = int(input())
```

One line, one integer. int() strips the newline for you.

"The second line contains n space-separated integers"

```
arr = list(map(int, input().split()))
```

One line again - n values do NOT mean n reads. Take the line, split on spaces, cast each piece.

THE FINISHED PARSE

```
n   = int(input())
arr = list(map(int, input().split()))

print(n, arr)        # echo: 5 [1, 2, 3, 4, 5]  -> correct
```

Remember: n and len(arr) are the same number. n is still read, because it is a line in the queue and skipping it would shift everything.

Walkthrough 2 - T test cases, two lines each

WHAT THE STATEMENT SAYS

```
Input Format

The first line contains T, the
number of test cases.
Each test case consists of
two lines:
 - two integers n and k
 - n space-separated integers
```

SAMPLE INPUT

```
2
5 2
1 2 3 4 5
3 1
9 8 7
```

TRANSLATING IT, LINE BY LINE

"The first line contains T"

```
t = int(input())
```

Read once, outside the loop. This is not part of any test case.

"Each test case consists of two lines"

```
for _ in range(t):
```

The two reads below go inside this loop, and they repeat exactly T times.

"two integers n and k"

```
n, k = map(int, input().split())
```

Two values on ONE line. One read, then unpack. Reading twice here is the most common mistake in this shape.

"n space-separated integers"

```
arr = list(map(int, input().split()))
```

Same as walkthrough 1, now nested inside the loop.

THE FINISHED PARSE

```
t = int(input())
for _ in range(t):
    n, k = map(int, input().split())
    arr = list(map(int, input().split()))

    print(n, k, arr)     # echo each case while you build it
    # ... solve this test case, print its answer
```

Remember: Print an answer per test case, inside the loop. T cases means T lines of output, not one.

Walkthrough 3 - a grid, and the split() decision

WHAT THE STATEMENT SAYS

```
Input Format

The first line contains two
integers R and C.
The next R lines each contain
C space-separated integers.
```

SAMPLE INPUT

```
3 3
1 0 1
0 1 0
1 1 1
```

TRANSLATING IT, LINE BY LINE

"two integers R and C"

```
r, c = map(int, input().split())
```

One line, two values, unpack.

"The next R lines"

```
for _ in range(r):
```

R is a loop count. This is where "the next N lines" always leads.

"C space-separated integers"

```
list(map(int, input().split()))
```

Each row is one line. Collect the rows into a list of lists.

THE FINISHED PARSE

```
r, c = map(int, input().split())
grid = [list(map(int, input().split())) for _ in range(r)]

print(grid)          # echo: [[1,0,1], [0,1,0], [1,1,1]]
# grid[i][j] is now row i, column j
```

Remember: If the grid is characters with no spaces (#.# rather than 1 0 1) then do NOT split - use grid = [input().strip() for _ in range(r)]. Splitting a spaceless row gives a one-element list and grid[i][j] quietly breaks.

Every input shape

Do not just read these. Cover the right column, look at the raw input, and type the parse yourself. A shape is learned when you can write it from blank without hesitating.

01 One integer

RAW STDIN	THE PARSE
`5`	`n = int(input())`

The simplest possible case. int() ignores the trailing newline for you, so no strip needed.

02 A count, then that many numbers on one line

RAW STDIN	THE PARSE
`5` `1 2 3 4 5`	`n = int(input())` `arr = list(map(int, input().split()))`

The single most common shape in Indian OAs. Note that n is often redundant — len(arr) gives you the same thing — but you MUST still read that first line or everything after it shifts by one.

03 Numbers on one line, no count given

RAW STDIN	THE PARSE
`1 2 3 4 5`	`arr = list(map(int, input().split()))` `n = len(arr)`

If you assume a count line that is not there, your first value gets eaten and every answer is wrong. Read the sample input character by character before you write anything.

04 Two or three values on one line

RAW STDIN	THE PARSE
`5 3`	`n, k = map(int, input().split())`

Unpacking straight from map() is the idiom. For three values: n, m, k = map(int, input().split())

05 A count, then that many separate lines

RAW STDIN	THE PARSE

`3`
`apple`
`banana`
`cherry`
	`n = int(input())`
	`items = [input().strip() for _ in range(n)]`

strip() is not optional here. Without it every string ends in \n, so 'apple\n' != 'apple' and your comparisons silently fail. This is one of the most common zero-scorers.

06 T test cases

RAW STDIN	THE PARSE

`2`
`3`
`1 2 3`
`4`
`5 6 7 8`
	`t = int(input())`
	`for _ in range(t):`
	`  n = int(input())`
	`  arr = list(map(int, input().split()))`
	`  print(solve(n, arr))`

Very common on CodeChef, HackerEarth and TCS. Print the answer for each case as you go, or collect them and join once at the end — the second is faster when T is large.

07 A grid of numbers

RAW STDIN

```
3 3
1 0 1
0 1 0
1 1 1
```

THE PARSE

```
r, c = map(int, input().split())
grid = [list(map(int, input().split())) for _ in range(r)]
```

Every matrix and island problem arrives like this. Watch the row/column order: it is almost always rows first, but read the statement rather than assuming.

08 A grid of characters

RAW STDIN

```
3 3
#.#
.#.
###
```

THE PARSE

```
r, c = map(int, input().split())
grid = [input().strip() for _ in range(r)]
```

No split() here — the characters are not separated by spaces. Using split() would give you one string per row inside a list, which quietly breaks grid[i][j] indexing.

09 A graph as an edge list

RAW STDIN

```
4 3
1 2
2 3
3 4
```

THE PARSE

```
n, m = map(int, input().split())
adj = [[] for _ in range(n + 1)]
for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v)
    adj[v].append(u)   # drop this line if directed
```

Size the list n+1 because these problems are almost always 1-indexed. For a weighted graph read u, v, w and append (v, w). Getting 1-indexing wrong is the classic off-by-one in every graph OA.

10 A string that contains spaces

RAW STDIN

```
the quick brown fox
```

THE PARSE

```
s = input().rstrip('\n')
```

Do NOT call split() here — it would destroy the sentence you were asked to process. If you used sys.stdin.readline you must strip, because readline keeps the newline.

11 Mixed types on one line

RAW STDIN

```
Kushal 21 88.5
```

THE PARSE

```
parts = input().split()
name = parts[0]
age = int(parts[1])
score = float(parts[2])
```

map(int, ...) would crash on the name. Split first, then cast each field individually.

12 Read until end of input

RAW STDIN

```
1 2
3 4
5 6
```

THE PARSE

```
import sys

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    a, b = map(int, line.split())
    print(a + b)
```

No count, no sentinel — just keep reading until the stream ends. Guard against blank trailing lines or you crash on the last iteration.

13 Read until a sentinel value

RAW STDIN

```
5
7
9
-1
```

THE PARSE

```
import sys

nums = []
for line in sys.stdin:
    x = int(line)
    if x == -1:
        break
    nums.append(x)
```

The sentinel is not part of the data. Break before appending, not after.

14 Everything at once (the heavy-input template)

RAW STDIN

```
3
1 2 3
4 5 6
7 8 9
```

THE PARSE

```
import sys

data = sys.stdin.buffer.read().split()
idx = 0

def nxt():
    global idx
    idx += 1
    return data[idx - 1]

n = int(nxt())
rows = [[int(nxt()) for _ in range(3)] for _ in range(n)]
```

When input is huge (10^5 lines or more) and line-by-line reading times out, read the whole stream once and walk a token pointer. This ignores line breaks entirely, which is usually fine and sometimes essential. Note buffer.read() gives bytes — int() handles them, but strings need .decode().

Output - where correct code still scores zero

The grader compares your output character by character against the expected output. It has no idea what you meant. **Print the answer and nothing else** - no labels, no prompts, no decoration.

This scores 0 out of 7 test cases

```
print("Enter a number:")    # never - there is no human
print("Output:", ans)       # the grader wanted just: 42
print("The answer is", ans)  # same problem
```

TCS states it in its own instructions: other than the answer, **nothing** should be written to stdout. The prompt that makes your program friendly to a human is what fails you.

The four output forms you need

```
print(ans)                              # a single value

print(*arr)                             # a list, space-separated
print(" ".join(map(str, arr)))          # identical, and explicit

print("\n".join(map(str, arr)))         # one per line

out = []                                # many test cases: collect,
for ... :                               # then flush once at the end.
    out.append(str(ans))                # much faster than printing
print("\n".join(out))                   # inside the loop
```

Case matters, and the statement decides

`YES`, `Yes` and `yes` are three different answers. So are `True` and `true`. Copy the exact spelling out of the problem statement rather than typing what feels natural.

Two Python hazards LeetCode has never shown you

LeetCode's limits are generous and its driver handles input for you. Neither of these has had a chance to bite you. Both will, on a real assessment platform.

1. The recursion limit is 1000

A perfectly correct DFS crashes with RecursionError on a 10^5-node tree or a large grid, because Python caps recursion depth at 1000 by default. You see a runtime error and assume your logic is wrong. It is not.

```
import sys
sys.setrecursionlimit(300000)    # before ANY recursive solution
```

Raising the limit alone can still blow the operating system's stack. For genuinely deep recursion, run on a thread with a bigger stack:

```
import sys, threading

def main():
    ...                          # your whole solution here

sys.setrecursionlimit(300000)
threading.stack_size(1 << 26)    # 64 MB of stack
t = threading.Thread(target=main)
t.start()
t.join()
```

Worth knowing for interviews too. Being able to say *"Python caps recursion at 1000, so for a tree this deep I would write it iteratively or raise the limit"* reads as experience.

2. input() in a loop times out

The built-in input() is slow. Past roughly 10^5 lines it alone can exceed the time limit even when your algorithm is optimal. HackerEarth deliberately includes maximum-constraint cases to catch this.

```
import sys
input = sys.stdin.readline      # one line, fixes the whole program
```

It works on my machine but scored zero

Run down this list in order. In an assessment the cause is almost always in the top four.

1. **You printed a label or a prompt.** Anything other than the answer itself fails the comparison.
2. **You did not strip the newline off a string.** 'apple\n' does not equal 'apple', so comparisons silently fail.
3. **The case is wrong.** YES vs Yes vs yes. Check the statement, not your instinct.
4. **You assumed a count line that was not there, or missed one that was.** Everything after it shifts by one and every answer is wrong.
5. **You are on Python 2.** Check the language dropdown. See the table on page 1.
6. **RecursionError on the large cases only.** Raise the recursion limit.
7. **Timeout on the large cases only.** Use sys.stdin.readline, and stop printing inside the loop.
8. **You read with input() when several values sat on one line.** One read takes the whole line; you then split it.
9. **Integer overflow.** Not a thing in Python - but if the statement says modulo 10^9+7, apply it as you go.
10. **Trailing whitespace or an extra blank line.** print() already adds exactly one newline; do not add another.

Where to practise, in order

Every link below was fetched and confirmed live. The whole ladder is about a week of evenings.

Do not use GeeksforGeeks Practice for this

This was checked directly: a GfG practice problem serves a **function stub** (`char* reverseString(char* s)`) with LeetCode-style examples. It is the same trap that got you at Infosys, so it cannot be the thing that fixes it. Same verdict for **IndiaBix** (no Python, no editor), **PrepInsta** (login wall) and **FACE Prep** (articles, no judge). GfG is fine for concepts - just not for this.

STEP 0 | TONIGHT | 30 MINUTES

See that the input box IS stdin

Free, no login, and it has an explicit Input box labelled "If your code takes input, add it in the above box before running". Paste each of the 14 raw samples above into that box and write the parse. That box is the same mechanism as the custom-input tab in a real assessment.

CodeChef online Python compiler
`https://www.codechef.com/python-online-compiler`
No login. Python 3. A real stdin box. Start here.

STEP 1 | DAYS 1-2

Blank-editor drills, where reading input IS the problem

HackerRank's Python practice domain is the no-stub kind: an empty editor, and the statement treats reading input as your job. Day 0 literally says "save a line of input from stdin", and Day 1 fails you if you hardcode the sample values instead of reading them.

30 Days of Code - Day 0: Hello World
`https://www.hackerrank.com/challenges/30-hello-world/problem`
The single best first problem. Reading one line from stdin is the entire task.

30 Days of Code - Day 1: Data Types
`https://www.hackerrank.com/challenges/30-data-types/problem`

Three typed values. Refuses hardcoded answers by design.

Python - Introduction
`https://www.hackerrank.com/domains/python/py-introduction`
Short set. Do all of it.

Python If-Else
`https://www.hackerrank.com/challenges/py-if-else/problem`
One integer in, one decision out. Shape 1.

What's Your Name?
`https://www.hackerrank.com/challenges/whats-your-name/problem`
Two lines of string input - the strip() lesson, learned the hard way. Shape 5.

Python - Basic Data Types
`https://www.hackerrank.com/domains/python/py-basic-data-types`
Shapes 2, 3 and 4.

Python practice domain (all of it)
`https://www.hackerrank.com/domains/python`
The full track if you want more reps.

STEP 2 | DAYS 3-5

CodeChef - stdin-native, no stub to fall back on

The beginner Python path is 192 problems, and modules 4, 5 and 14 are literally named *User inputs*, *simple problems on input, output and basic math*, and *test cases & custom input/output*. That last one is the T-test-cases shape, which is shape 6 and constant on TCS and HackerEarth.

CodeChef Practice - Python path
`https://www.codechef.com/practice/python`
192 problems. Modules 4, 5 and 14 are the ones you came for.

CodeChef Practice - classic problemset
`https://www.codechef.com/practice-old`
5000+ problems, sortable by difficulty ascending. For once the shapes are automatic.

STEP 3 | DAY 6

HackerEarth - and deliberately feel one timeout

HackerEarth hosts the Flipkart, Swiggy, Razorpay and Sprinklr assessments, and it is competitive-programming style by design. It also sets maximum-constraint cases on purpose. Do one problem with input() in a loop and let it time out, then change that single line to sys.stdin.readline and watch it pass. Learning it once by experience beats reading it ten times.

Basics of Input/Output - practice problems
`https://www.hackerearth.com/practice/basic-programming/input-output/basics-of-input-output/practice-problems/`
20 problems, pure stdin. Free to read; free account to submit.

Basics of Input/Output - tutorial
`https://www.hackerearth.com/practice/basic-programming/input-output/basics-of-input-output/tutorial/`
Read this first if any shape still feels shaky.

STEP 4 | DAY 7

Now look at the stubbed side, so you can name the difference

These hand you a read-only header that already reads stdin and a footer that already prints - you fill only the function body, exactly like LeetCode. Open one and click the arrow to expand the read-only header. Seeing the parsing code that was being written for you all along is the moment it makes sense. In a real OA your first question is then always: am I in a stubbed editor or a blank one?

Solve Me First
`https://www.hackerrank.com/challenges/solve-me-first/problem`
The stubbed kind. Expand the header and read what it does for you.

Simple Array Sum

`https://www.hackerrank.com/challenges/simple-array-sum/problem`

Statement describes a count line and an array line - but the stub eats both, not you.

Interview Preparation Kit

`https://www.hackerrank.com/interview/interview-preparation-kit`

Stubbed throughout. The mode Amazon, Salesforce and Adobe use.

Reference, for when something goes wrong

Codeforces - Python input template

`https://codeforces.com/blog/entry/71884`

A copy-paste template with four helper functions. Three minutes to read.

Codeforces - fast input with os.read

`https://codeforces.com/blog/entry/83441`

The heavy-input trick, with real timed-out-then-accepted submissions.

USACO Guide - fast I/O

`https://usaco.guide/general/fast-io`

Benchmarks it: 18.9 seconds down to 2.9, from one changed line.

HackerRank - Reading Raw Input tutorial

`https://www.hackerrank.com/challenges/python-raw-input/tutorial`

raw_input() against input() side by side - literally the Python 2 vs 3 confusion above.

Codeforces - recursion depth and the threading trick

`https://codeforces.com/blog/entry/80158`

The working stack-size workaround, plus a decorator alternative.

Python docs - sys.setrecursionlimit

`https://docs.python.org/3/library/sys.html#sys.setrecursionlimit`

The primary source.

Python docs - threading.stack_size

`https://docs.python.org/3/library/threading.html#threading.stack_size`

Pairs with the above for genuinely deep recursion.

The OOP trap for a Python-only candidate

You said companies focus on DSA and OOP. That is **true for interviews and mostly false for the assessment stage** at the mass recruiters. Checking current published patterns for Accenture, Infosys, TCS NQT, Wipro NLTH, Capgemini and Cognizant: not one has a dedicated OOP MCQ section. Their OAs are aptitude, then pseudocode or output-tracing, then coding. OOP turns up inside "programming fundamentals" MCQs, in Cognizant GenC's Python cluster, and then heavily in the interview that follows.

You can know Python OOP perfectly and still fail the MCQ

Essentially every OOP question bank online is written in **C++ or Java syntax**, and several concepts they test either do not exist in Python or behave differently - `virtual` functions, `interface`, `private` access modifiers, and compile-time method overloading.

The fix is **not** to switch languages. You need C++/Java as a *reading* skill - enough to trace what a snippet does - while you keep writing Python. Budget six to eight focused hours, and do it after the input work.

C++ OOP quizzes - GeeksforGeeks
`https://www.geeksforgeeks.org/cpp/cpp-oops-quizzes/`
The C++-syntax grind. This is the reading skill; do the most reps here.

OOP concepts quiz - GeeksforGeeks
`https://www.geeksforgeeks.org/quizzes/oop-concepts-gq/`
Language-agnostic concept questions - the four pillars, dispatch, abstraction.

OOPs MCQs - InterviewBit
`https://www.interviewbit.com/oops-mcq/`
Mixed bank, closest in style to what an actual OA serves.

Python OOP exercises - PyNative
`https://pynative.com/python-object-oriented-programming-oop-exercise/`
Python-native practice, for the interview rather than the MCQ.

Python inheritance MCQ - GeeksforGeeks
`https://www.geeksforgeeks.org/quizzes/python-inheritance-mcq/`
Fills the gap between the two above.